DJANGO WEB APPLICATION • TASK 7

PRODUCTION READY

Student Registration Portal

User Authentication, Secure Sessions & Relational Academic Records

PROJECT NAME	APP NAME	SECURITY	DATABASE
task7	**myuser**	**PBKDF2 Password Hashing**	**SQLite3 (Relational)**

Executive Summary: Task 7 is a full-featured Django authentication and academic intake portal. It features custom user registration with password cryptography (`make_password/check_password`), session-based user authentication, and a protected multi-tenant student registration dashboard where authenticated staff members enroll students and inspect live record archives.

1. Data Architecture & Relational Models (`myuser/models.py`)

The system establishes a one-to-many relational schema between administrative portal users and registered student entities.

Model	Field	Type / Constraints	Description
PortalUser	`username`	CharField(max_length=20, unique=True)	Unique credential identity for login
	`password`	CharField(max_length=128)	Secure PBKDF2 cryptographic hashed digest
	`name`	CharField(max_length=50)	Full display name of portal officer
StudentRegistration	`student_id`	CharField(max_length=20, unique=True)	Unique institutional student identification number
	`student_name`	CharField(max_length=100)	Student candidate legal name
	`semester`	CharField(max_length=30)	Enrolled academic term/semester
	`subject`	CharField(max_length=100)	Academic subject or major branch
	`registered_by`	ForeignKey(PortalUser, CASCADE)	Relational foreign key tying student records to creator

```python
from django.db import models


class PortalUser(models.Model):
    username = models.CharField(max_length=20, unique=True)
    password = models.CharField(max_length=128)
    name = models.CharField(max_length=50)

    def __str__(self):
        return self.username


class StudentRegistration(models.Model):
    student_id = models.CharField(max_length=20, unique=True)
    student_name = models.CharField(max_length=100)
    semester = models.CharField(max_length=30)
    subject = models.CharField(max_length=100)
    registered_by = models.ForeignKey(
        PortalUser,
        on_delete=models.CASCADE,
        related_name='student_registrations',
    )

    def __str__(self):
        return f'{self.student_id} - {self.student_name}'
```

2. Application Routing & Endpoints

Routing is divided cleanly between project-level routing (`task7/urls.py`) and app-level endpoint mapping (`myuser/urls.py`).

Route Endpoint	Handler Method	Name	Access Policy
`''`	`views.home`	`'home'`	Public (Shows dynamic state if logged in)
`'sign-up/'`	`views.sign_up`	`'sign_up'`	Public Registration Form
`'sign-up/save/'`	`views.save_user`	`'save_user'`	POST Handler with Duplicate Check
`'login/'`	`views.login`	`'login'`	Public Authentication Entrypoint
`'login/validate/'`	`views.login_vel`	`'login_vel'`	POST Verification & Session Initiation
`'students/register/'`	`views.student_registration`	`'student_registration'`	Protected (Session verification required)
`'logout/'`	`views.logout_view`	`'logout'`	Flushes session and redirects to home

App-Level URLs (`myuser/urls.py`):

```python
from django.urls import path

from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('sign-up/', views.sign_up, name='sign_up'),
    path('sign-up/save/', views.save_user, name='save_user'),
    path('login/', views.login, name='login'),
    path('login/validate/', views.login_vel, name='login_vel'),
    path('students/register/', views.student_registration, name='student_registration'),
    path('logout/', views.logout_view, name='logout'),
]
```

Project-Level URLs (`task7/urls.py`):

```python
"""
URL configuration for task7 project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/6.1/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.contrib import admin
from django.urls import include, path

```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('myuser.urls')),
]

from django.contrib.staticfiles.urls import staticfiles_urlpatterns
urlpatterns += staticfiles_urlpatterns()
```

3. View Controller & Business Logic (`myuser/views.py`)

The controller manages stateful interactions, session keys, credential verification, and CRUD operations on student records.

> **Key Architectural Highlights:**
>
> - `_current_user(request)`: Helper function retrieving active `PortalUser` from session ID safely.
> - `save_user(request)`: Validates fields, checks for duplicate usernames, and hashes password using PBKDF2 before saving.
> - `login_vel(request)`: Authenticates input against hash digest using `check_password` and establishes session.
> - `student_registration(request)`: Enforces authentication guard, processes POST registrations, and queries user-specific student records.

```python
from django.contrib.auth.hashers import check_password, make_password
from django.shortcuts import redirect, render
from .models import PortalUser,StudentRegistration

SESSION_USER_ID = 'task7_portal_user_id'


def _current_user(request):
    user_id = request.session.get(SESSION_USER_ID)
    if user_id is None:
        return None
    return PortalUser.objects.filter(pk=user_id).first()


def home(request):
    return render(request, 'home.html', {'portal_user': _current_user(request)})


def sign_up(request):
    return render(request, 'sign_up.html')


def save_user(request):
    if request.method != 'POST':
        return redirect('sign_up')

    username = request.POST.get('username', '').strip()
    name = request.POST.get('name', '').strip()
    password = request.POST.get('password', '')

    if not username or not name or not password:
        return render(request, 'sign_up.html', {
            'errormsg': 'Please fill in all fields.'
        })
    if PortalUser.objects.filter(username=username).exists():
        return render(request, 'sign_up.html', {
            'errormsg': 'That username is already in use.'
        })

    PortalUser.objects.create(
        username=username,
        name=name,
        password=make_password(password),
    )
    return redirect('login')


def login(request):
    return render(request, 'login.html')


def login_vel(request):
    if request.method != 'POST':
```

```
        return redirect('login')

    username = request.POST.get('username', '').strip()
    password = request.POST.get('password', '')
    portal_user = PortalUser.objects.filter(username=username).first()

    if portal_user is None or not check_password(password, portal_user.password):
        return render(request, 'login.html', {
            'errormsg': 'Incorrect username or password.'
        })

    request.session[SESSION_USER_ID] = portal_user.pk
    return redirect('student_registration')


def student_registration(request):
    portal_user = _current_user(request)
    if portal_user is None:
        return redirect('login')

    if request.method == 'POST':
        student_id = request.POST.get('student_id', '').strip()
        student_name = request.POST.get('student_name', '').strip()
        semester = request.POST.get('semester', '').strip()
        subject = request.POST.get('subject', '').strip()

        if not all((student_id, student_name, semester, subject)):
            students = StudentRegistration.objects.filter(
                registered_by=portal_user
            ).order_by('student_name')
            return render(request, 'student_registration.html', {
                'portal_user': portal_user,
                'students': students,
                'errormsg': 'Please fill in all fields.',
            })

        if StudentRegistration.objects.filter(student_id=student_id).exists():
            students = StudentRegistration.objects.filter(
                registered_by=portal_user
            ).order_by('student_name')
            return render(request, 'student_registration.html', {
                'portal_user': portal_user,
                'students': students,
                'errormsg': 'That student ID is already registered.',
            })

        StudentRegistration.objects.create(
            student_id=student_id,
            student_name=student_name,
            semester=semester,
            subject=subject,
            registered_by=portal_user,
        )
        request.session['task7_registration_saved'] = True
        return redirect('student_registration')

    students = StudentRegistration.objects.filter(
        registered_by=portal_user
    ).order_by('student_name')
    saved = request.session.pop('task7_registration_saved', False)
    return render(request, 'student_registration.html', {
        'portal_user': portal_user,
        'students': students,
        'saved': saved,
    })


def logout_view(request):
    request.session.flush()
    return redirect('home')
```

4. Live Application Output Screenshots

The screenshots below document all primary application interfaces captured live with active session states and database data.

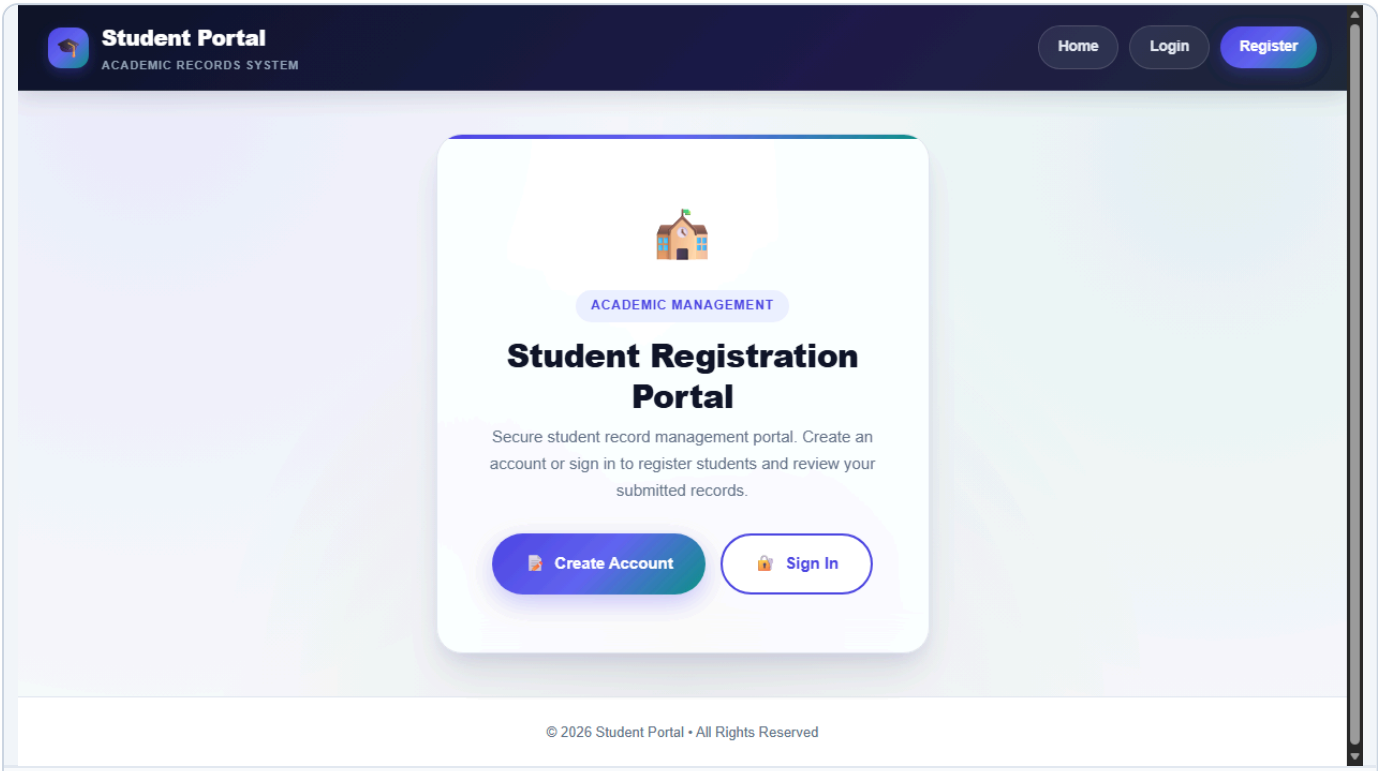


Figure 2.1: Portal Landing & Home Interface featuring navigation to registration, login, and system overview.

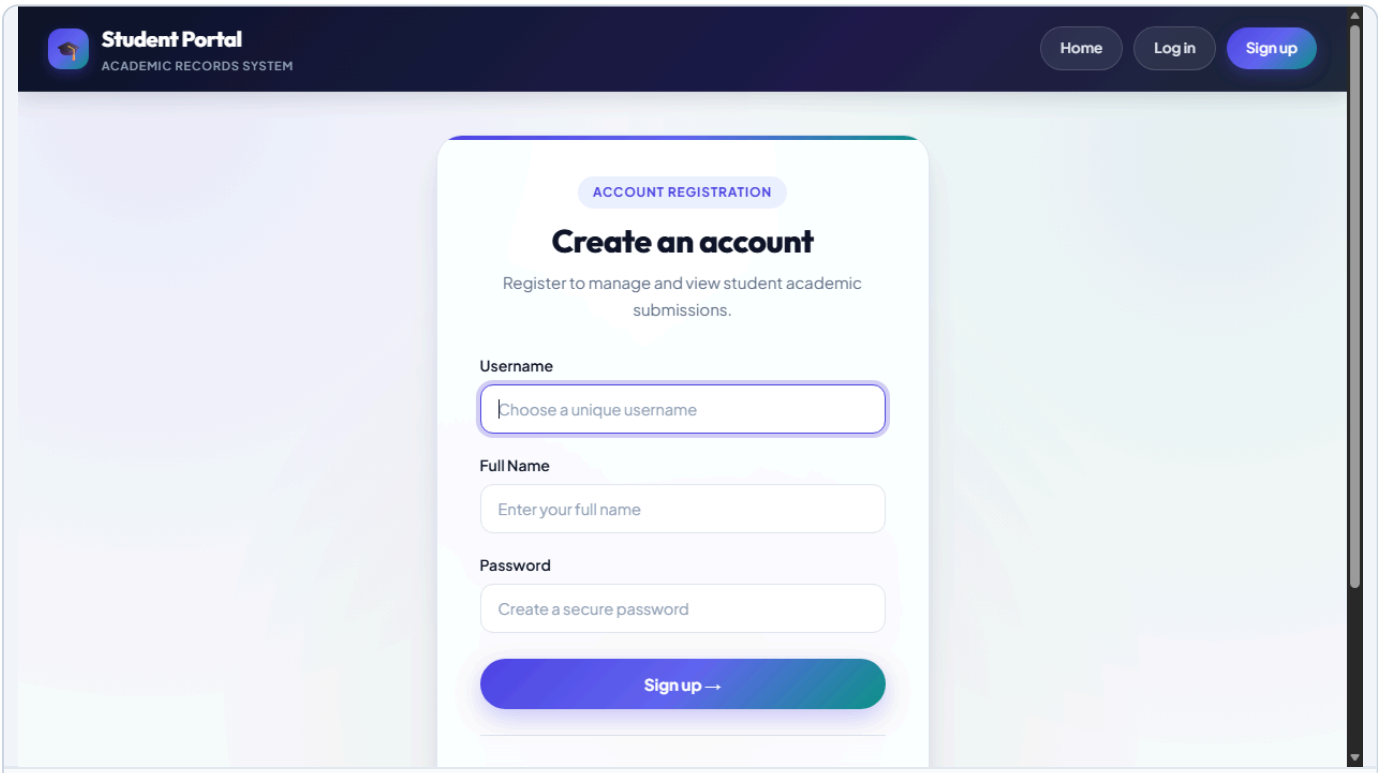


Figure 2.2: Officer Registration Portal with full name, username, and encrypted password configuration.

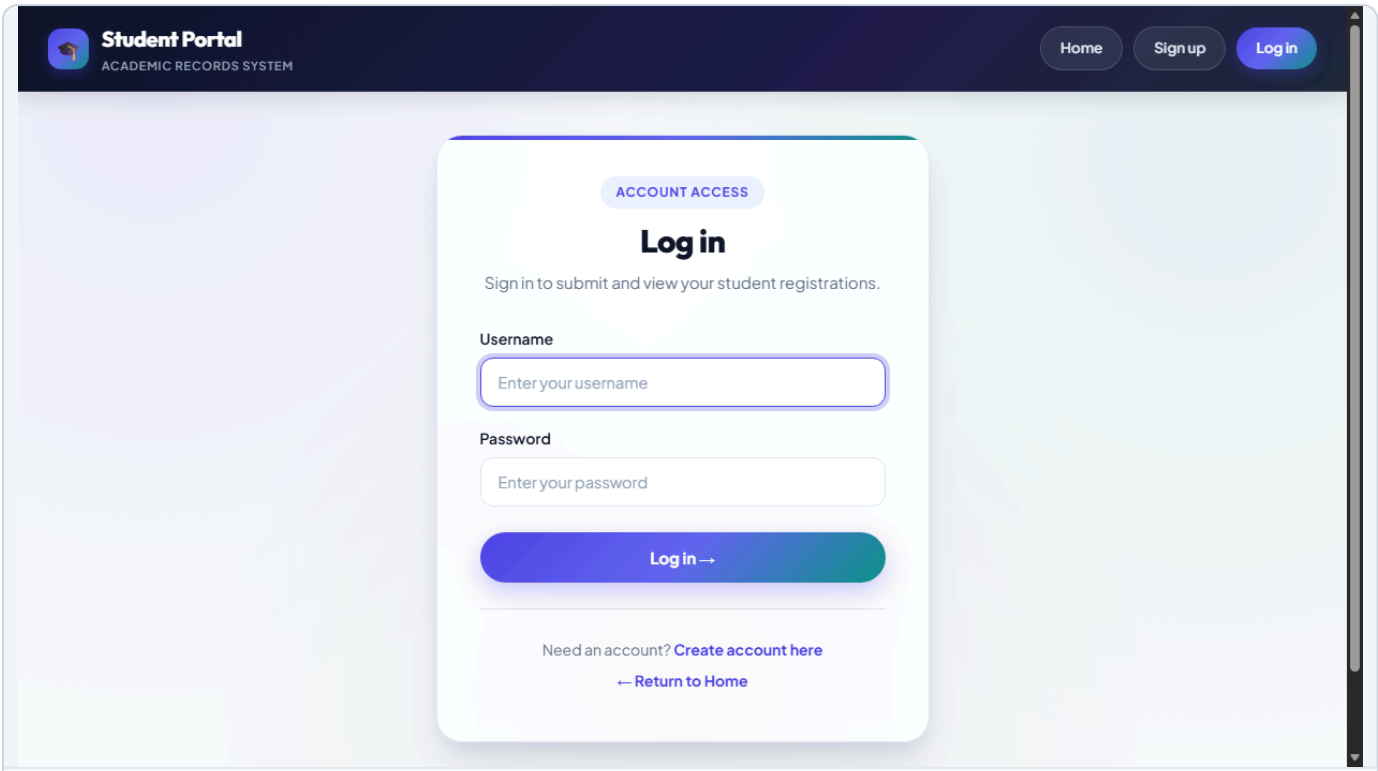


Figure 2.3: Secure User Authentication Portal verifying cryptographic hashes against SQLite database.

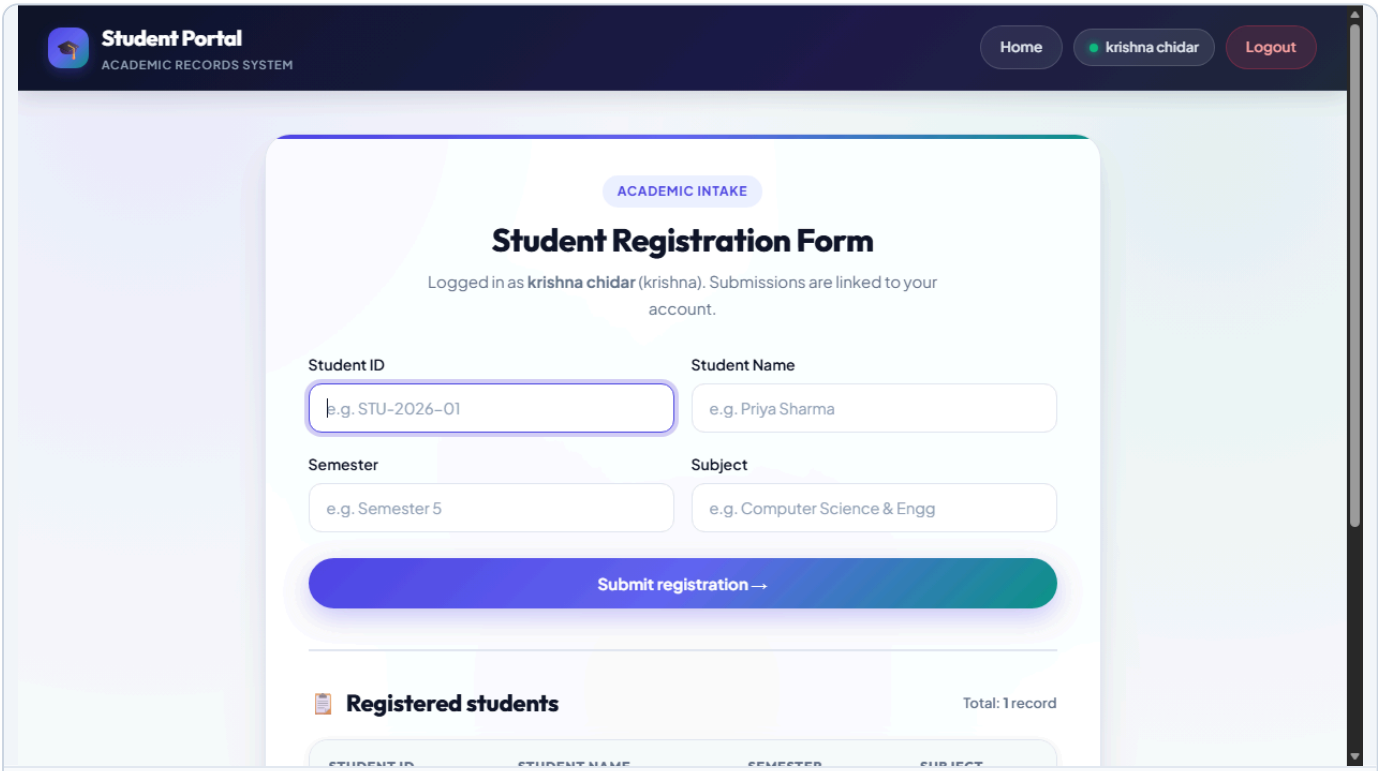


Figure 2.4: Protected Student Intake Dashboard displaying student registration form and live table of enrolled student records linked to logged-in officer *krishna chidar*.